

FastAPI Project Structure & Data Flow — From Beginner to Production

A learning and interview-revision guide. It explains how a FastAPI project is organized and how data moves through it, BEFORE you build a real application. It is not a complete app.

How to read this guide

There is no single universal FastAPI folder structure. FastAPI itself does not force any layout. The layouts below are popular conventions that solve real problems (messy code, hard testing, tangled responsibilities). Your team, your app size and your requirements decide what you actually use.

Throughout the guide, ideas are tagged so you know how strict they are:

Tag	Meaning
[Required]	FastAPI / Python really needs this to work
[Common practice]	Most teams do it, for good reasons, but you can choose otherwise
[Optional]	Add it only when the project actually needs it

Learning order: concepts first → small project → growing project → production flows → AI/RAG. Every section builds on the previous one.

1. FastAPI Fundamentals

1.1 What FastAPI is

FastAPI is a Python **web framework for building APIs**. An API is a program that other programs (a React app, a mobile app, another server) talk to over HTTP, usually exchanging JSON.

FastAPI is built on two libraries:

- **Starlette** – handles the web part (requests, responses, routing, middleware)
- **Pydantic** – handles data validation using Python type hints

Why people like it: you write normal Python functions with type hints, and FastAPI gives you request validation, JSON conversion and interactive docs (`/docs`) automatically. It runs on an ASGI server such as **Uvicorn**.

1.2 Endpoint / route

A **route** (or endpoint) connects a **URL path + HTTP method** to a **Python function**.

```
from fastapi import FastAPI

app = FastAPI()

@app.get('/health')          # method = GET, path = /health
def health_check():
    return {'status': 'ok'}  # becomes JSON automatically
```

Read it as: "When a client sends `GET /health`, run `health_check` and send back what it returns."

1.3 Request vs response

	Request (client → server)	Response (server → client)
Line	Method + path: <code>GET /users?limit=10</code>	Status code: <code>200 OK</code> , <code>404 Not Found</code>
Headers	<code>Authorization: Bearer <token></code> , <code>Content-Type</code>	<code>Content-Type:</code> <code>application/json</code>
Body	JSON data (for POST/PUT/PATCH)	JSON data returned

Useful status codes: `200 OK` · `201 Created` · `204 No Content` · `400 Bad request` · `401 Not authenticated` · `403 Not allowed` · `404 Not found` · `422 Validation failed` · `500 Server error`.

1.4 HTTP methods

Method	Purpose	Example	Notes
GET	Read data	<code>GET /users/5</code>	Never changes data
POST	Create something new	<code>POST /users</code>	Body holds new data; return <code>201</code>

Method	Purpose	Example	Notes
PUT	Replace a whole resource	<code>PUT /users/5</code>	Client sends the full object
PATCH	Change part of a resource	<code>PATCH /users/5</code>	Client sends only changed fields
DELETE	Remove a resource	<code>DELETE /users/5</code>	Often returns <code>204</code>

```

@app.post('/users', status_code=201)
def create_user(...): ...

@app.patch('/users/{user_id}')
def update_user(user_id: int, ...): ...

@app.delete('/users/{user_id}', status_code=204)
def delete_user(user_id: int): ...

```

1.5 Pydantic

Pydantic lets you describe the *shape* of data as a class. FastAPI uses it to check incoming data and shape outgoing data.

```

from pydantic import BaseModel, EmailStr

class UserCreate(BaseModel):
    email: EmailStr
    password: str
    full_name: str | None = None

@app.post('/users')
def create_user(payload: UserCreate):
    # By the time we get here, payload is already valid.
    return {'email': payload.email}

```

If the client sends `{"email": "not-an-email"}`, FastAPI automatically answers **422** with an explanation. Your function never runs with bad data.

Mental model: a Pydantic class is a *contract* for data crossing the API boundary.

1.6 Dependency Injection (Depends)

Many functions need the same helpers: a database session, the current user, settings. Instead of creating them inside every function, you **declare** what you need and FastAPI **provides** it.

```
from fastapi import Depends

def get_db():
    db = SessionLocal()
    try:
        yield db          # give the session to the route
    finally:
        db.close()        # always cleaned up afterwards

@app.get('/users')
def list_users(db = Depends(get_db)):
    ...
```

Why it matters: shared setup lives in one place, cleanup is automatic, and in tests you can swap the real database for a fake one.

1.7 SQLAlchemy

SQLAlchemy is the most common Python **ORM** (Object-Relational Mapper). It lets you work with database tables as Python classes instead of writing raw SQL everywhere.

```
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase): pass

class User(Base):
    # = table 'users'
    __tablename__ = 'users'
    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(unique=True)
    hashed_password: Mapped[str]
```

- A **model** = one table described as a class.
- A **session** = a short conversation with the database (query, add, commit).
- Query example: `db.query(User).filter(User.email == email).first()`

A **SQLAlchemy model** (database shape) is NOT the same thing as a **Pydantic schema** (API shape). Section 3 explains why.

1.8 Middleware

Middleware is code that runs around *every* request, before it reaches a route and after the response leaves it.

```
import time

@app.middleware('http')
async def add_timing(request, call_next):
    start = time.time()
    response = await call_next(request)      # run the route
    response.headers['X-Process-Time'] = str(time.time() - start)
    return response
```

Typical uses: **CORS** (letting your React app call the API), logging, timing, request IDs.

	Middleware	Dependency
Applies to	Every request	Only routes that ask for it
Good for	CORS, logging, timing	DB session, current user, permissions

1.9 Authentication / authorization (quick preview)

- **Authentication** = *Who are you?* (login, password, token)
- **Authorization** = *What are you allowed to do?* (roles, ownership)

In modern APIs, the user logs in once, receives a **JWT** (a signed token), and sends it in the `Authorization: Bearer <token>` header on later requests. Section 5 covers the full flow.

2. A Small FastAPI Project

Start here. For a prototype or a tiny API, this is *enough*.

```
project/
├─ main.py
├─ requirements.txt
└─ .env
```

Purpose of every file

File	Purpose	Required?
<code>main.py</code>	Creates the <code>FastAPI()</code> app, defines routes, is the entry point Uvicorn runs	[Required] (file name is a convention; any name works)
<code>requirements.txt</code>	Lists Python packages so anyone can run <code>pip install -r requirements.txt</code>	[Common practice] (also <code>pyproject.toml</code>)
<code>.env</code>	Holds secrets and settings (database URL, API keys) outside the code	[Common practice] . Never commit it to Git; add it to <code>.gitignore</code>

main.py

```
from fastapi import FastAPI

app = FastAPI(title='Small API')

FAKE_USERS = [{'id': 1, 'name': 'Asha'}, {'id': 2, 'name': 'Ravi'}]

@app.get('/users')
def list_users():
    return FAKE_USERS
```

requirements.txt

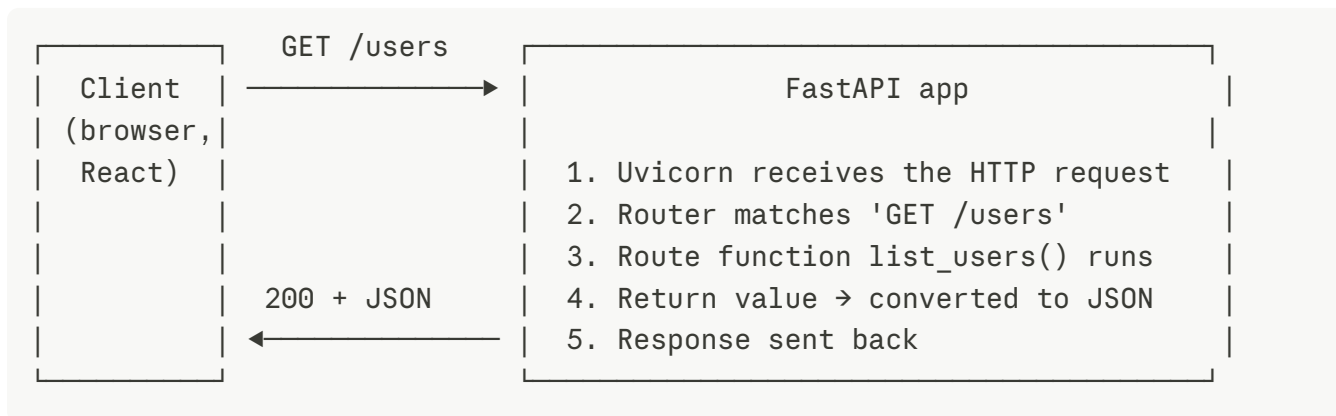
```
fastapi
uvicorn[standard]
python-dotenv
```

.env

```
DATABASE_URL=postgresql://user:password@localhost:5432/mydb
SECRET_KEY=change-me
```

Run it: `uvicorn main:app --reload` → open `http://127.0.0.1:8000/docs`.

Complete request flow: GET /users



In words:

1. **Client** sends `GET /users`.
2. **FastAPI** (through Uvicorn) receives it and looks at the route table.
3. The **route** `@app.get('/users')` matches.
4. The **function** `list_users()` runs and returns a Python list.
5. FastAPI turns it into JSON and sends a **response** with status `200`.

When this small structure stops being enough

You are outgrowing it when you notice: `main.py` is hundreds of lines, you scroll to find things, tests are hard to write, or two people keep editing the same file. That is the signal to move to Section 3 — not before.

3. A Growing FastAPI Project

When the app has real features (users, documents, chats), one file is not enough. The idea is simple: **separate code by its job**, so each file has one reason to change.

```

app/
├─ main.py
├─ api/           ← HTTP endpoints (routes)
├─ schemas/       ← Pydantic: data entering/leaving the API
├─ models/        ← SQLAlchemy: database tables
├─ services/      ← business logic (what the app does)
├─ repositories/  ← database access (queries)
├─ core/          ← config, security, DB setup
└─ tests/         ← automated tests
  
```

[Common practice], not mandatory. FastAPI does not require any of these folders. Some teams merge `services` and `repositories`; some group by feature (`users/` , `documents/`) instead of by layer. Pick the simplest structure that keeps your code readable.

`main.py` — the start button

Creates the app, registers middleware (CORS) and plugs in the routers. It should stay **tiny** (about 20–40 lines): no business logic, no queries.

```
from fastapi import FastAPI
from app.api import users, documents

app = FastAPI(title='KnowledgeHub')
app.include_router(users.router, prefix='/users', tags=['users'])
app.include_router(documents.router, prefix='/documents', tags=['documents'])
```

`api/` — HTTP endpoints

1. Why it exists: The outside world talks to your app through HTTP. This folder is the *front door*: it receives requests and returns responses, nothing more.

2. File types: One router file per resource: `users.py` , `auth.py` , `documents.py` , `chat.py` . Also shared dependencies such as `deps.py` (`get_db` , `get_current_user`).

3. What belongs here: Route decorators, path/query parameters, declaring the request schema and response schema, status codes, calling **one service function**, translating errors into HTTP errors (`HTTPException`).

4. What does NOT belong here: Business rules ("a free user can have at most 5 documents"), SQL or ORM queries, password hashing, calls to Gemini or the vector DB.

5. Example:

```
# app/api/users.py
router = APIRouter()

@router.post('/', response_model=UserRead, status_code=201)
def register(payload: UserCreate, db: Session = Depends(get_db)):
    return user_service.register_user(db, payload)    # route delegates
```

The route is thin: *receive* → *delegate* → *return*.

schemas/ — the API contract (Pydantic)

- 1. Why it exists:** You need a precise definition of what data the API accepts and returns, separate from how the database stores it. Schemas validate input and control what is exposed.
- 2. File types:** Pydantic classes grouped by resource: `user.py`, `document.py`, `chat.py`.
- 3. What belongs here:** `UserCreate` (input), `UserRead` (output), `UserUpdate` (partial input), field rules (min length, email format), `from_attributes` config so schemas can be built from ORM objects.
- 4. What does NOT belong here:** Database columns/relationships, queries, business logic, anything that talks to a database.
- 5. Example:**

```
# app/schemas/user.py
class UserCreate(BaseModel):
    email: EmailStr
    password: str = Field(min_length=8)

class UserRead(BaseModel):
    id: int
    email: EmailStr          # note: NO password field here
    model_config = ConfigDict(from_attributes=True)
```

The client sends a password but never gets one back. That safety comes from having separate input and output schemas.

models/ — the database shape (SQLAlchemy)

- 1. Why it exists:** Your database tables need a Python description so SQLAlchemy can create tables and run queries.
- 2. File types:** SQLAlchemy classes: `user.py`, `document.py`, `chunk.py`, `message.py`, plus a shared `base.py`.
- 3. What belongs here:** Table names, columns, types, foreign keys, relationships, indexes, constraints.
- 4. What does NOT belong here:** HTTP validation rules, request/response shapes, business workflows, calls to external services.
- 5. Example:**

```
# app/models/document.py
class Document(Base):
    __tablename__ = 'documents'
    id: Mapped[int] = mapped_column(primary_key=True)
    owner_id: Mapped[int] = mapped_column(ForeignKey('users.id'))
    filename: Mapped[str]
    storage_path: Mapped[str]
    status: Mapped[str] = mapped_column(default='uploaded')
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
```

services/ — business logic (what the app does)

- 1. Why it exists:** Real apps have *rules* and *workflows* that are neither HTTP nor SQL. Keeping them in one place makes them reusable (an API route, a background job and a CLI can call the same function) and easy to test.
- 2. File types:** One service per domain: `user_service.py`, `auth_service.py`, `document_service.py`, `rag_service.py`.
- 3. What belongs here:** Business rules, multi-step workflows, coordinating several repositories or external services, decisions ("is this email already taken?", "does the user own this document?").
- 4. What does NOT belong here:** `Request` / `Response` objects, `HTTPException` (that is HTTP-specific; raise your own errors instead), raw SQL, FastAPI imports.
- 5. Example:**

```
# app/services/user_service.py
def register_user(db, payload: UserCreate):
    if user_repo.get_by_email(db, payload.email):
        raise EmailAlreadyExists() # a business rule
    hashed = hash_password(payload.password)
    return user_repo.create(db, payload.email, hashed)
```

Notice: the service *decides*, the repository *stores*.

repositories/ — database access

- 1. Why it exists:** Without it, queries get scattered across routes and services, and changing the database logic means hunting through the whole code base. A repository is the **only place that talks to the database**.
- 2. File types:** One repository per model: `user_repo.py`, `document_repo.py`, `message_repo.py`.

3. What belongs here: CRUD functions (`get_by_id` , `get_by_email` , `create` , `list_for_user` , `delete`), query filters, pagination, joins.

4. What does NOT belong here: Business rules, HTTP concerns, password hashing, calls to external APIs.

5. Example:

```
# app/repositories/user_repo.py
def get_by_email(db: Session, email: str) -> User | None:
    return db.query(User).filter(User.email == email).first()

def create(db: Session, email: str, hashed_password: str) -> User:
    user = User(email=email, hashed_password=hashed_password)
    db.add(user); db.commit(); db.refresh(user)
    return user
```

[Optional] For small apps, a separate repository layer can be extra ceremony. Many projects start by putting queries in services and extract repositories when queries grow. Both are valid.

core/ — shared infrastructure & configuration

1. Why it exists: Some things are needed everywhere and are not features: settings, database connection, security helpers, logging. They live in one dependable place.

2. File types: `config.py` (settings from `.env`), `database.py` (engine + session), `security.py` (hashing, JWT), `logging.py` , `exceptions.py` .

3. What belongs here: Loading environment variables, creating the DB engine and `get_db` , password hashing and token creation/decoding helpers, custom exception classes.

4. What does NOT belong here: Feature logic ("upload a document"), routes, feature-specific queries.

5. Example:

```
# app/core/config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str
    secret_key: str
    access_token_minutes: int = 30
    model_config = {'env_file': '.env'}

settings = Settings()  # reads .env once; the rest of the app imports this
```

tests/ — does it work?

- 1. Why it exists:** Tests let you change code with confidence. They also prove each layer works on its own.
- 2. File types:** `test_*.py` files, mirroring the app: `test_users_api.py`, `test_user_service.py`, `test_user_repo.py`, and a `conftest.py` for shared fixtures (test client, test database).
- 3. What belongs here:** API tests (using `TestClient`), service tests (business rules, often with a fake repository), repository tests (against a real test database), fixtures.
- 4. What does NOT belong here:** Production code, real secrets, tests that depend on your real production database or on live paid APIs (mock Gemini instead).
- 5. Example:**

```
# app/tests/test_user_service.py
def test_register_rejects_duplicate_email(db):
    register_user(db, UserCreate(email='a@b.com', password='password1'))
    with pytest.raises(EmailAlreadyExists):
        register_user(db, UserCreate(email='a@b.com', password='password1'))
```

3.1 Four distinctions that confuse every beginner

Schema vs Model

	Schema (schemas/)	Model (models/)
Library	Pydantic	SQLAlchemy
Describes	Data crossing the API boundary	Data stored in the database

	Schema (<code>schemas/</code>)	Model (<code>models/</code>)
Question it answers	"What may the client send/see?"	"What does the table look like?"
Typical fields	<code>password</code> (input only), no <code>hashed_password</code>	<code>hashed_password</code> , <code>created_at</code> , foreign keys
Can differ per use?	Yes: <code>UserCreate</code> , <code>UserRead</code> , <code>UserUpdate</code>	One class per table

Why separate? The database has fields clients must never see (`hashed_password`), and the API often has fields the database does not store (`password`, `confirm_password`). One class cannot safely be both.

Service vs Repository

	Service	Repository
Thinks about	<i>What should happen?</i>	<i>How do I read/write the data?</i>
Knows about	Rules, workflows, other services	Tables, queries, sessions
Example	"Reject duplicate emails, hash the password, then save"	"SELECT a user WHERE email = ..."
Rule of thumb	Decides	Fetches and stores

Route vs Service

	Route (<code>api/</code>)	Service (<code>services/</code>)
Speaks	HTTP: status codes, headers, JSON	Plain Python: functions and objects
Job	Receive, validate (via schema), delegate, respond	Apply rules, coordinate work
Reusable outside web?	No	Yes (jobs, scripts, tests)
Smell	Route has 40 lines of <code>if</code> logic	Service imports <code>Request</code> or returns <code>JSONResponse</code>

Configuration vs Business Logic

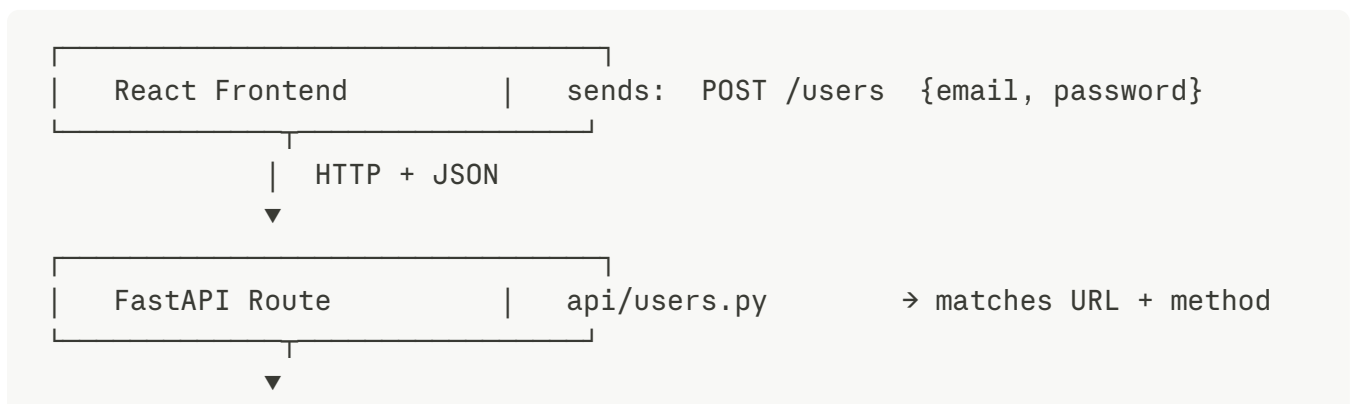
	Configuration (<code>core/</code>)	Business logic (<code>services/</code>)
What	Values that change per environment	Rules that define the product
Examples	Database URL, secret key, token lifetime, model name	"Max 10 documents per user", "only owners can delete"
Changed by	Deployment / <code>.env</code>	A developer, via code review
Rule	Never hardcode secrets in code	Never read raw env vars inside logic; use <code>settings</code>

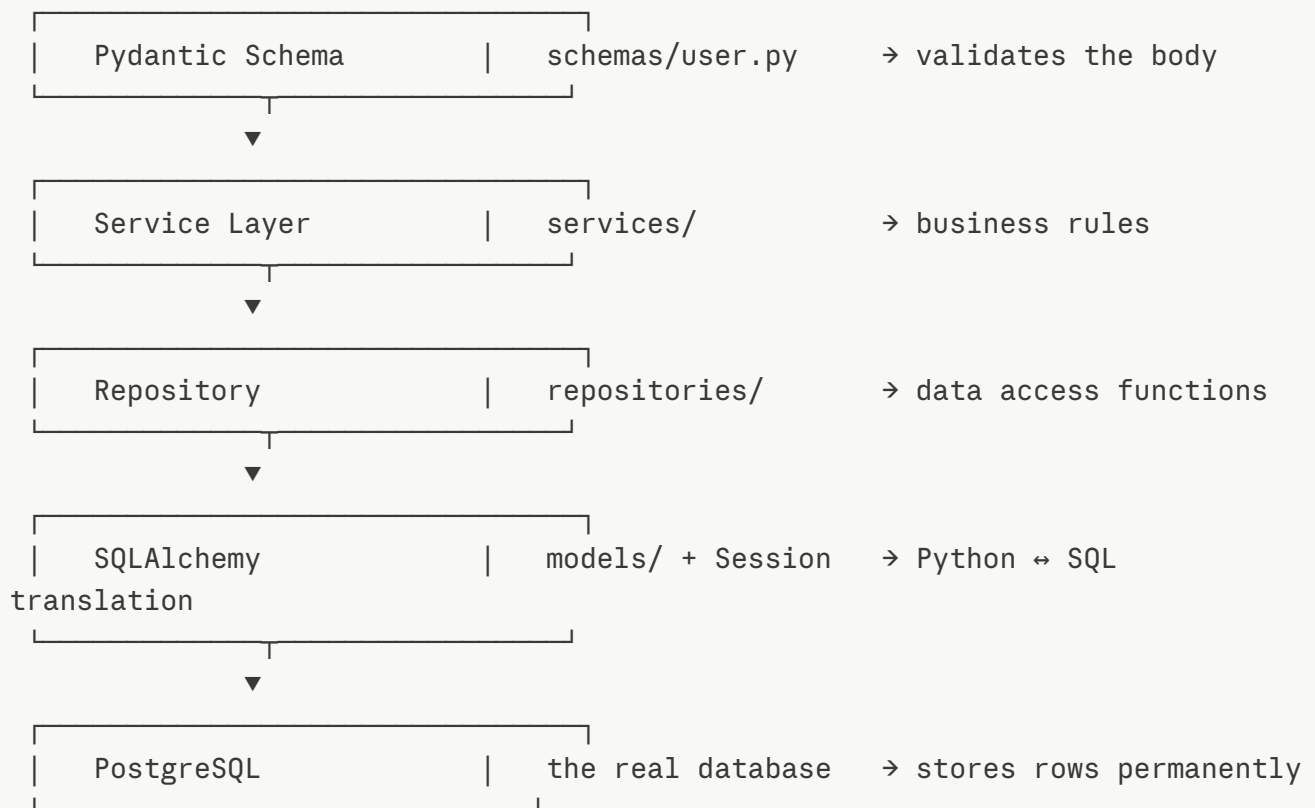
Following one feature through the layers

Step	Layer	Responsibility
1	Route	<code>POST /users</code> received
2	Schema	Body validated as <code>UserCreate</code>
3	Service	Duplicate check, hash password
4	Repository	<code>INSERT INTO users ...</code>
5	Model	Defines the <code>users</code> table that INSERT targets
6	Schema	Result shaped as <code>UserRead</code> (no password)
7	Route	<code>201 Created</code> returned

4. Production Request Flow

This is the backbone of almost every production FastAPI app. Learn this one diagram well.





The response travels back UP the same path:

PostgreSQL row → Model object → Service result → Response schema → JSON → React

What happens at each stage

#	Stage	What happens	Example
1	React	User clicks "Sign up". The app sends an HTTP request with a JSON body.	<code>fetch('/users', {method: 'POST', body: ...})</code>
2	Route	FastAPI finds the function registered for <code>POST /users</code> .	<code>@router.post('/')</code>
3	Schema	The body is checked against <code>UserCreate</code> . Bad data is rejected with <code>422</code> before any of your logic runs.	Email format, password length
4	Service	Applies business rules and coordinates the work.	"Email must be unique", "hash the password"

#	Stage	What happens	Example
5	Repository	Runs the data-access call. The only layer that knows how data is queried.	<code>user_repo.create(...)</code>
6	SQLAlchemy	Converts Python objects and method calls into SQL and manages the session/transaction.	<code>db.add(user); db.commit()</code>
7	PostgreSQL	Executes the SQL, enforces constraints (unique, foreign key), stores the row durably.	<code>INSERT INTO users ...</code>

On the way back, the created `User` model object is converted into the `UserRead` schema, so only safe fields are serialized to JSON.

Where do errors go? A service raises a domain error (`EmailAlreadyExists`). The route (or a global exception handler) converts it into an HTTP response (`409 Conflict`). This keeps the service free of HTTP details.

5. Authentication Flow

5.1 Authentication vs authorization

	Authentication (AuthN)	Authorization (AuthZ)
Question	<i>Who are you?</i>	<i>What are you allowed to do?</i>
Happens	First	After authentication
Based on	Password, token, API key	Roles, permissions, resource ownership
Failure code	<code>401 Unauthorized</code>	<code>403 Forbidden</code>
Example	Valid JWT for user #5	User #5 may delete <i>their own</i> documents, but not user #6's

Airport analogy: **authentication** is the passport check (identity). **authorization** is whether your boarding pass lets you into the lounge.

5.2 Login flow

```

Client (React)
  | POST /login  {email, password}
  ▼
Route  (api/auth.py)
  |
  ▼
Schema validation  (LoginRequest: is it a valid email? password present?)
  |
  ▼
Auth Service  (services/auth_service.py)
  | "find this user, check the password, issue a token"
  ▼
User Repository  (repositories/user_repo.py)
  | get_by_email(email)
  ▼
PostgreSQL → returns the user row (including hashed_password)
  |
  ▼
Password verification  (core/security.py: verify_password(plain, hashed))
  | fail → 401 Unauthorized
  ▼
JWT generation  (core/security.py: create_access_token(user_id))
  |
  ▼
Response → {access_token: '...', token_type: 'bearer'}

```

Key ideas:

- Passwords are stored as a **hash** (e.g. bcrypt/argon2), never as plain text. You cannot reverse a hash; you hash the login attempt and compare.
- Return the **same vague error** for "unknown email" and "wrong password" (`Invalid credentials`) so attackers cannot discover which emails exist.
- A **JWT** is a signed string containing claims such as `sub` (user id) and `exp` (expiry). The server signs it with `SECRET_KEY` ; anyone can read it, but nobody can forge it without the key. Do not put secrets inside it.

```
# core/security.py (shape only)
def create_access_token(user_id: int) -> str:
    payload = {'sub': str(user_id), 'exp': now() + timedelta(minutes=30)}
    return jwt.encode(payload, settings.secret_key, algorithm='HS256')
```

5.3 Authenticated request flow

```
Client + header  →  Authorization: Bearer <JWT>
| GET /documents
▼
FastAPI dependency  (Depends(get_current_user))
|
▼
JWT verification  (signature valid? not expired?)
| fail → 401
▼
Current User  (load user by id from the 'sub' claim)
|
▼
Route  (receives current_user as a parameter)
|
▼
Service  (document_service.list_documents(db, current_user))
```

```
# api/deps.py
def get_current_user(token: str = Depends(oauth2_scheme), db =
Depends(get_db)):
    user_id = decode_token(token)          # raises if invalid/expired
    user = user_repo.get_by_id(db, user_id)
    if not user:
        raise HTTPException(status_code=401)
    return user

# api/documents.py
@router.get('/')
def list_my_documents(user = Depends(get_current_user), db =
Depends(get_db)):
    return document_service.list_documents(db, user)
```

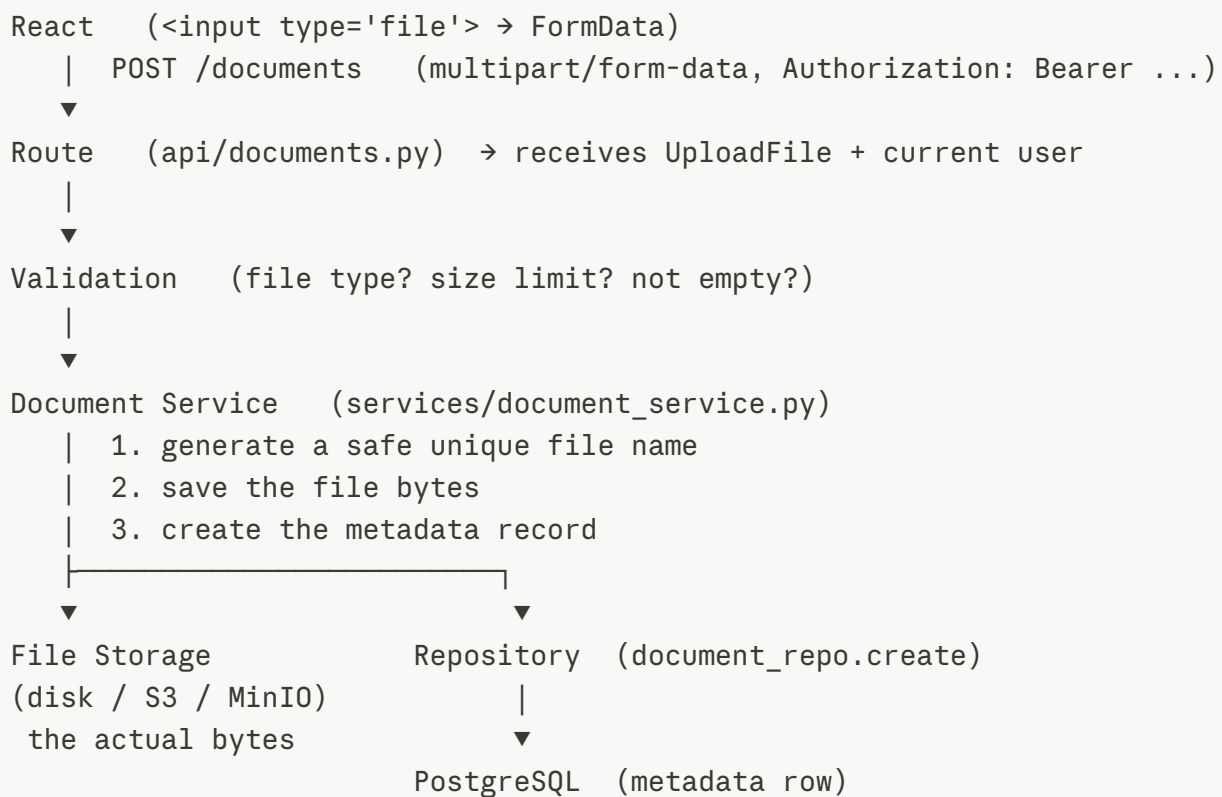
The route never parses tokens itself. **One dependency authenticates; every protected route just adds `Depends(get_current_user)`.**

5.4 Where authorization lives

Authentication gives you *who*. Authorization checks happen in the **service layer**, because they are business rules:

```
# services/document_service.py
def delete_document(db, user, document_id):
    doc = document_repo.get(db, document_id)
    if doc is None:
        raise NotFound()
    if doc.owner_id != user.id and user.role != 'admin':    # authorization
rule
        raise Forbidden()
    document_repo.delete(db, doc)
```

6. File Upload Flow



Where does what go?

Data	Where	Why
Metadata (id, owner, original filename, size, MIME type, storage path, status, created_at)	PostgreSQL	Small, structured, needs queries, filters, ownership checks
The actual file (PDF, DOCX, image bytes)	File storage: local disk (dev), S3/GCS/MinIO or a Docker volume (production)	Files are large binary blobs; databases are poor and expensive for storing them

The database row simply **points to** the file via `storage_path`.

```
# api/documents.py
@router.post('/', response_model=DocumentRead, status_code=201)
async def upload(file: UploadFile, user = Depends(get_current_user), db =
Depends(get_db)):
    return await document_service.upload_document(db, user, file)

# services/document_service.py (shape only)
async def upload_document(db, user, file):
    validate_file(file)                # type + size
    path = await storage.save(file)     # bytes → storage
    return document_repo.create(db, owner_id=user.id, filename=file.filename,
                                storage_path=path, status='uploaded')
```

Production tips:

- Never trust the client's filename; generate your own (e.g. a UUID) to avoid overwrites and path tricks.
- Check size and type on the server; frontend checks are only a convenience.
- Store an explicit `status` (`uploaded` → `processing` → `ready` → `failed`). This becomes essential for AI document processing in Section 7.
- Wrap storage behind a small interface (`storage.save` , `storage.read`) so moving from local disk to S3 does not touch the rest of the app.

7. AI / RAG Application Architecture

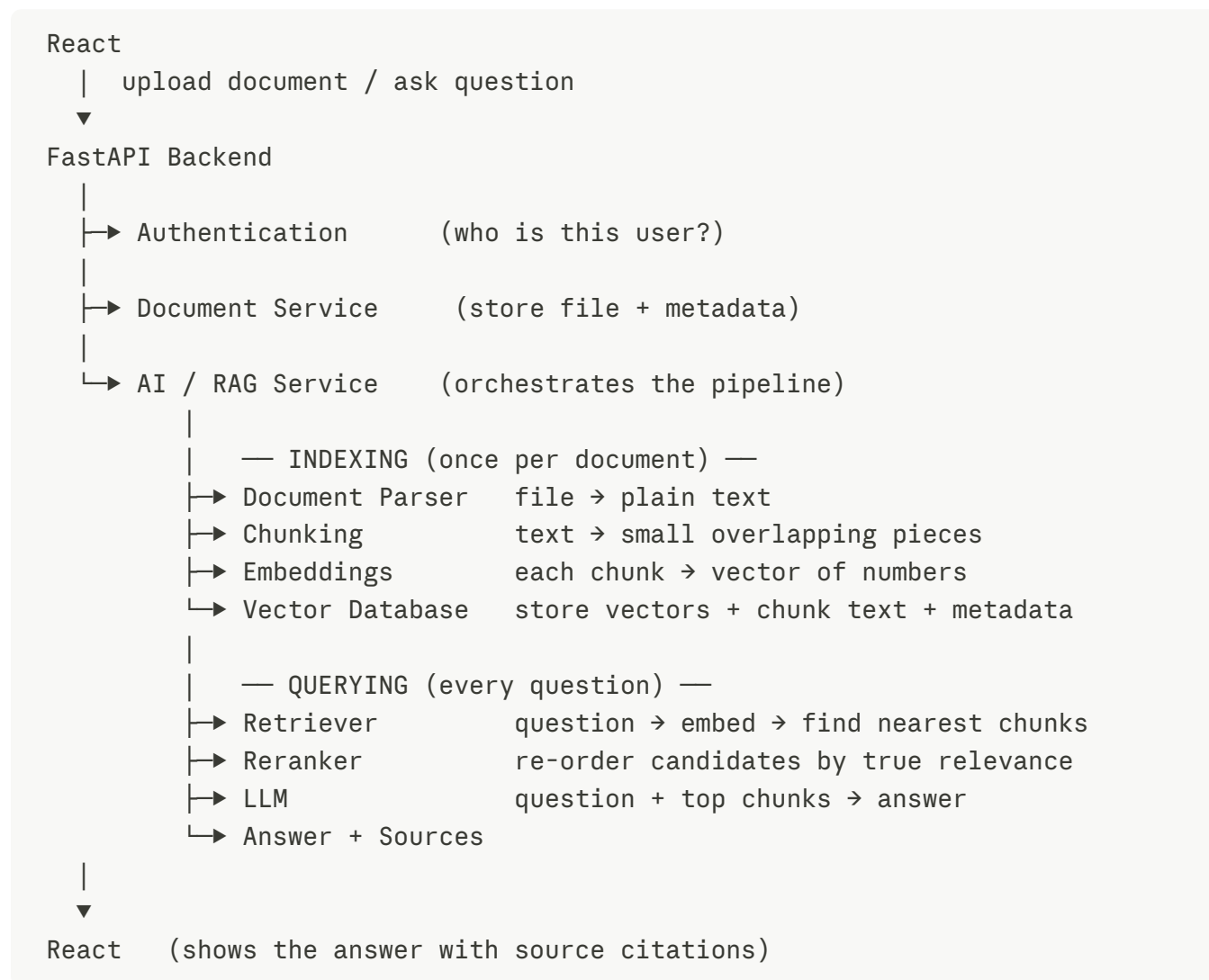
7.1 What RAG is (in plain words)

RAG = Retrieval-Augmented Generation. An LLM only knows what it was trained on; it has never seen *your* documents. RAG fixes this in two steps:

1. **Retrieve** the few passages of your documents that are relevant to the question.
2. **Generate** an answer by giving the LLM those passages as context.

The LLM is the writer; your application is the librarian who fetches the right pages first.

7.2 The full architecture



7.3 Pipeline A — Indexing (document ingestion)

Runs **once per document**, usually in the background because it can be slow.

PDF / DOCX / TXT



Parser → extracts clean text (and page numbers)



Chunker → splits into ~300-800 token pieces with some overlap



Embedder → each chunk becomes a vector, e.g. [0.12, -0.80, ...]



Vector DB → stores: vector + chunk text + {document_id, page, owner_id}

Step	Why it exists
Parser	Models need text, not PDF bytes. Parsing quality directly limits answer quality.
Chunking	Whole documents are too big to send to an LLM and too vague to search. Small chunks give precise matches. Overlap prevents cutting an idea in half.
Embeddings	An embedding model turns text into numbers so that <i>similar meaning = nearby vectors</i> . This enables search by meaning, not just keywords.
Vector DB	Built to find "the nearest vectors" quickly among millions.

7.4 Pipeline B — Query / chat flow

Runs **every time the user asks a question**.

```

React: 'What is our refund policy?'
▼
FastAPI route → authenticate user → validate schema
▼
RAG Service (the application is in control from here)
|
| ▶ 1. Embed the question          (same embedding model as indexing!)
| ▶ 2. Retriever: vector search    (top 20 chunks, filtered by owner_id)
| ▶ 3. Reranker: re-score the 20   (keep best 4-5)
| ▶ 4. Build the prompt:
|       'Answer ONLY using the context below. Cite sources.
|       Context: [chunk 1] [chunk 2] ...
|       Question: ...'
| ▶ 5. Call the LLM (Gemini)       → generates the answer
| ▶ 6. Save messages to PostgreSQL (conversation history)
▼
Response: { answer, sources: [{document, page, snippet}] }
▼
React displays the answer + clickable sources

```

Reranker, explained: vector search is fast but approximate. A reranker is a slower, more accurate model that looks at (*question, chunk*) pairs and re-sorts them. Typical pattern: retrieve many (fast, rough) → rerank to a few (slow, precise). **[Optional]:** many RAG apps start without it and add it when answer quality needs improving.

7.5 Four things you must be able to say clearly

- 1. The LLM does NOT directly search the vector database.** The LLM is a text-in, text-out model. It has no connection to Qdrant or Chroma.
- 2. Your application performs retrieval.** Your code embeds the question, queries the vector DB and filters by user.
- 3. Retrieved chunks are given to the LLM as context.** They are pasted into the prompt. The LLM only sees what you put in the prompt.
- 4. PostgreSQL and the vector database have different jobs.** (Table below.)

	PostgreSQL	Vector Database (Qdrant / Chroma)
Stores	Users, documents (metadata), conversations, messages, processing status	Chunk embeddings + chunk text + small metadata payload

	PostgreSQL	Vector Database (Qdrant / Chroma)
Answers	"Which documents does user 5 own?" · "Show chat history"	"Which chunks are <i>semantically closest</i> to this question?"
Search type	Exact: filters, joins, IDs	Approximate nearest-neighbour by meaning
Source of truth for	Application data & permissions	Search index (can be rebuilt from the files)
Typical mistake	Trying to do semantic search with <code>LIKE</code>	Using it as the main database for users/chats

Note: PostgreSQL *can* also store vectors (pgvector). A separate vector DB is a common choice, not a requirement. For a small app, pgvector may be simpler. Pick based on scale and operational comfort.

8. KnowledgeHub AI — A Realistic Example

KnowledgeHub AI is a hypothetical app: users upload documents (policies, manuals, notes) and chat with an AI that answers using only those documents, with source citations.

8.1 Technology and responsibilities

Technology	Role in the system
React	UI: login, upload, document list, chat window, source viewer
FastAPI	Backend API and orchestration
PostgreSQL + SQLAlchemy	Users, documents metadata, conversations, messages, job status
Redis	Caching, rate limiting, and/or a job queue/broker for background processing
Qdrant / Chroma	Vector store for chunk embeddings (Qdrant = standalone server; Chroma = simpler, great for local/dev)

Technology	Role in the system
Hugging Face embeddings	Turn chunks and questions into vectors (e.g. a sentence-transformers model)
Gemini	The LLM that writes the final answer from retrieved context
Docker	Runs every service the same way on any machine (<code>docker compose up</code>)

8.2 One possible backend structure

Important: this is **one possible** architecture, not *the* correct one. Folder structures are architectural choices. They should change as requirements change: a small team may merge folders, a bigger one may split into feature modules or microservices. Treat this as a worked example, not a template to copy blindly.

```

knowledgehub-ai/
├── docker-compose.yml      # api, postgres, redis, qdrant (+ worker)
├── frontend/              # React app
└── backend/
    ├── Dockerfile
    ├── requirements.txt
    ├── .env
    ├── alembic/           # database migrations
    └── app/
        ├── main.py
        ├── api/
        │   ├── deps.py    # get_db, get_current_user
        │   ├── auth.py    # /register, /login
        │   ├── documents.py # upload, list, delete
        │   └── chat.py    # ask, conversation history
        ├── schemas/
        │   ├── user.py    document.py  chat.py
        ├── models/
        │   ├── user.py    document.py  conversation.py  message.py
        ├── services/
        │   ├── auth_service.py
        │   ├── document_service.py
        │   ├── chat_service.py
        │   └── rag/       # the AI sub-system lives together
        │       ├── rag_service.py    # orchestrates the flow
        │       └── parser.py

```

```

|       |— chunker.py
|       |— embeddings.py      # Hugging Face model wrapper
|       |— vector_store.py    # Qdrant/Chroma wrapper
|       |— retriever.py
|       |— reranker.py
|       |— llm.py             # Gemini wrapper + prompt building
|— repositories/
|   |— user_repo.py  document_repo.py  conversation_repo.py
|— core/
|   |— config.py  database.py  security.py  redis.py
|— tests/

```

8.3 Design decisions and the reasons behind them

Decision	Reason	Could be different if...
AI code grouped in <code>services/rag/</code>	RAG has many moving parts; keeping them together makes the pipeline easy to find and swap	The app is small → one <code>rag_service.py</code> is enough
Vector store, embeddings and LLM each wrapped in their own small file	You can swap Chroma→Qdrant or Gemini→another model by changing one file	You are certain you will never switch
Document processing runs in the background	Parsing + embedding can take seconds to minutes; the upload request should return immediately	Documents are tiny → process inline
Redis	Rate-limit chat requests, cache repeated answers/embeddings, hold a task queue	Low traffic → skip Redis at first
<code>owner_id</code> stored in vector metadata	Retrieval must only return the <i>current user's</i> chunks (security)	The knowledge base is shared by everyone
Docker Compose	One command to run API + Postgres + Redis + Qdrant	You deploy on a managed platform

9. Data Flow Examples (KnowledgeHub AI)

Each flow follows the same shape: **Frontend** → **API** → **Service** → **Repository / AI Service** → **Database / Vector DB** → **Response**.

A. User registration

```

React form {email, password}
  ▼ POST /auth/register
API route      validates body with UserCreate schema
  ▼
Auth Service   is the email taken? → hash password
  ▼
User Repository INSERT user
  ▼
PostgreSQL     users table gets a new row
  ▼
Response 201 {id, email}      (UserRead – never the password/hash)

```

B. Login

```

React {email, password}
  ▼ POST /auth/login
API route      LoginRequest schema
  ▼
Auth Service   fetch user → verify password hash → create JWT
  ▼
User Repository get_by_email
  ▼
PostgreSQL     returns the user row
  ▼
Response 200 {access_token, token_type}
React stores the token (in memory or a secure cookie) and sends it on later
requests

```

C. Upload document

```

React file + Bearer token
  ▼ POST /documents (multipart)
API route      get_current_user → validate file type/size
  ▼
Document Service  safe filename → save file → create record
(status='uploaded')
  ├── File Storage      saves the actual bytes
  └── Document Repository INSERT metadata
      ▼
      PostgreSQL      documents table row
  ▼ (also queues a background job: process_document(document_id))
Response  202/201 {id, filename, status:'uploaded'} ← returns fast

```

D. Process document into embeddings (background job)

```

Worker / background task  process_document(document_id)
  ▼
Document Service  set status = 'processing' (Repository → PostgreSQL)
  ▼
AI / RAG Service
  ├── Parser      read file from storage → text
  ├── Chunker     text → chunks
  ├── Embeddings  Hugging Face model → vectors
  └── Vector DB   upsert vectors + text + {document_id, owner_id, page}
      ▼
Document Service  set status = 'ready' (or 'failed' + error message)
  ▼
PostgreSQL  documents.status updated
React polls GET /documents/{id} (or receives a push) and shows 'Ready'

```

Why a background job? Parsing and embedding can take a long time. The HTTP request should not hang while it runs.

E. Ask a RAG question

```

React {conversation_id, question} + token
  ▼ POST /chat/ask
API route      authenticate → validate AskRequest schema
  ▼
Chat Service   check the conversation belongs to this user (authorization)
  ▼
AI / RAG Service
  |▶ embed question (Hugging Face)
  |▶ Retriever → Vector DB (filter owner_id = user.id) → top-k chunks
  |▶ Reranker → best few chunks
  |▶ build prompt = instructions + chunks + (recent history) + question
  |▶ Gemini → answer text
  ▼
Message Repository save user message + assistant answer + sources
  ▼
PostgreSQL      messages table
  ▼
Response {answer, sources:[{document, page, snippet}]}

```

Notice: **FastAPI code does the retrieval**, then hands the chunks to Gemini. Gemini never touches the vector DB.

F. Retrieve conversation history

```

React (opens a chat) + token
  ▼ GET /conversations/{id}/messages?limit=50
API route      authenticate
  ▼
Chat Service   verify ownership → ask repository for messages
  ▼
Message Repository SELECT ... WHERE conversation_id = ? ORDER BY
created_at
  ▼
PostgreSQL     (no vector DB involved – this is plain relational data)
  ▼
Response [{role:'user'|"assistant', content, sources, created_at}, ...]

```

This flow is a good reminder of the division of labour: **history = PostgreSQL, semantic search = vector DB**.

10. Common Beginner Mistakes

Mistake	Why it hurts	Better approach
Everything in <code>main.py</code>	Becomes unreadable, hard to test, merge conflicts	Split into routers, schemas, services as the app grows; keep <code>main.py</code> tiny
Business logic inside routes	Rules can't be reused or tested without HTTP; routes balloon	Route validates and delegates; service holds the rules
Mixing Pydantic schemas and SQLAlchemy models	Leaks internal fields (<code>hashed_password</code>); API changes force DB changes	Separate <code>schemas/</code> and <code>models/</code> ; convert between them
Database queries everywhere	Same query copied in many places; schema changes break everything	Centralize in repositories (or at least in services)
Hardcoding secrets	Keys leak via Git; can't change per environment	Environment variables / <code>.env</code> loaded by <code>core/config.py</code> ; never commit <code>.env</code> ; use a secrets manager in production
Letting the LLM make authentication decisions	LLMs can be tricked by prompts (prompt injection)	Auth is done by your code with JWT + checks. The LLM never decides who someone is or what they may access
Mixing RAG retrieval with UI logic	Frontend cannot be trusted; pipeline becomes impossible to change	Retrieval lives in the backend RAG service; React only sends questions and renders answers
Too many folders too early	Navigation overhead; empty layers add no value	Start small; split when a file hurts to work in
Unnecessary abstraction	Interfaces, factories and base classes with a single use make code harder to read	Add an abstraction only when you feel a <i>real</i> second use case

Mistake	Why it hurts	Better approach
Not testing service/database boundaries	Bugs hide at the seams: wrong query, wrong rule, wrong permission	Unit-test services with fake repos; integration-test repositories on a real test DB; API-test key routes

Related security habit for RAG: always filter retrieval by the current user's id (or permitted collection). Otherwise one user's question can pull another user's private chunks into the prompt.

11. Mental Model

When you open any FastAPI project, ask these questions folder by folder:

```

main.py          → How does the application start?

api/             → What HTTP endpoints exist?

schemas/        → What data enters and leaves the API?

models/         → What does the database look like?

services/       → What should the application DO?

repositories/   → How does the application ACCESS the database?

core/           → What shared infrastructure / configuration exists?

tests/          → Does it work correctly?

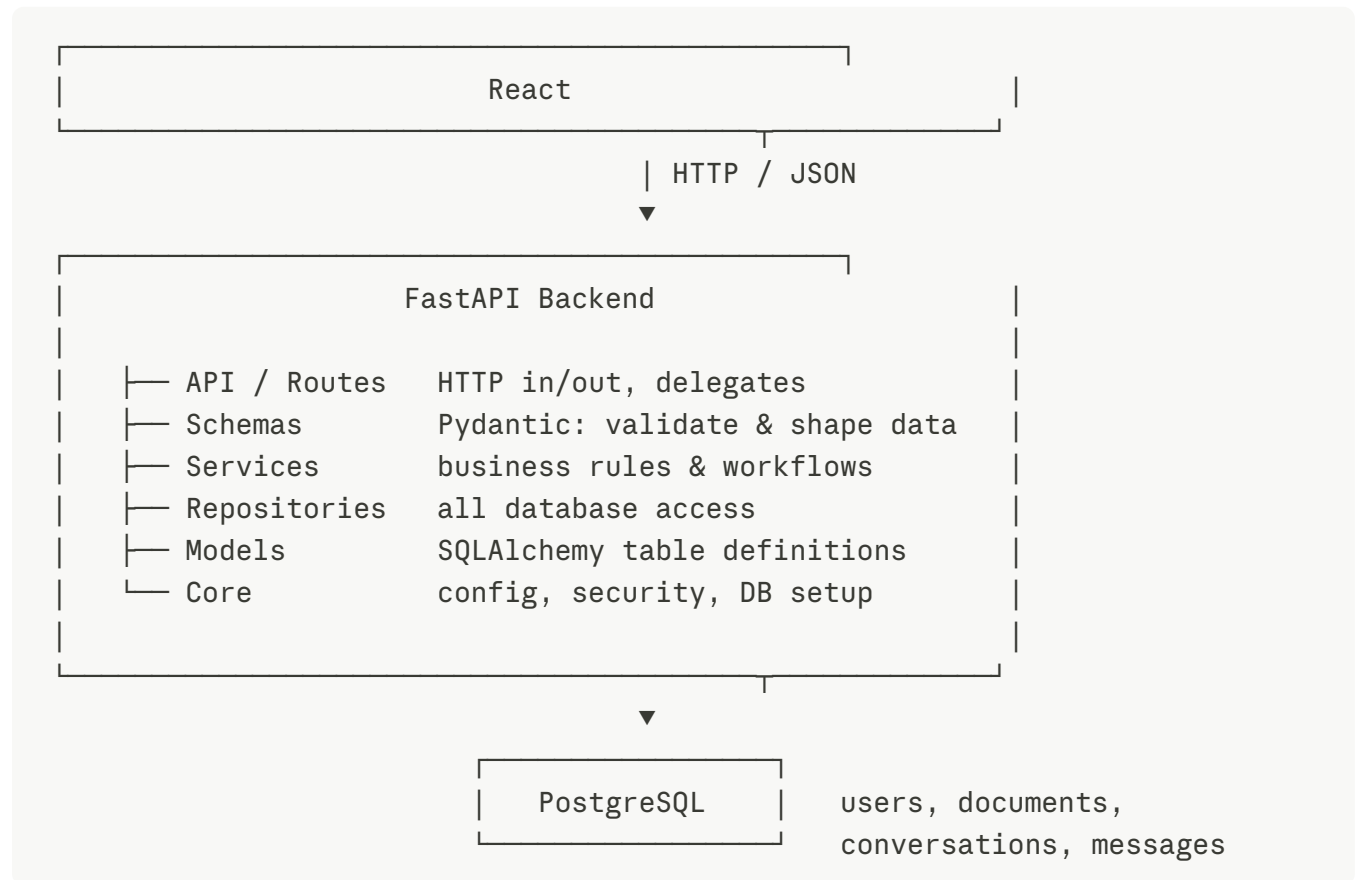
```

One-line versions for interviews:

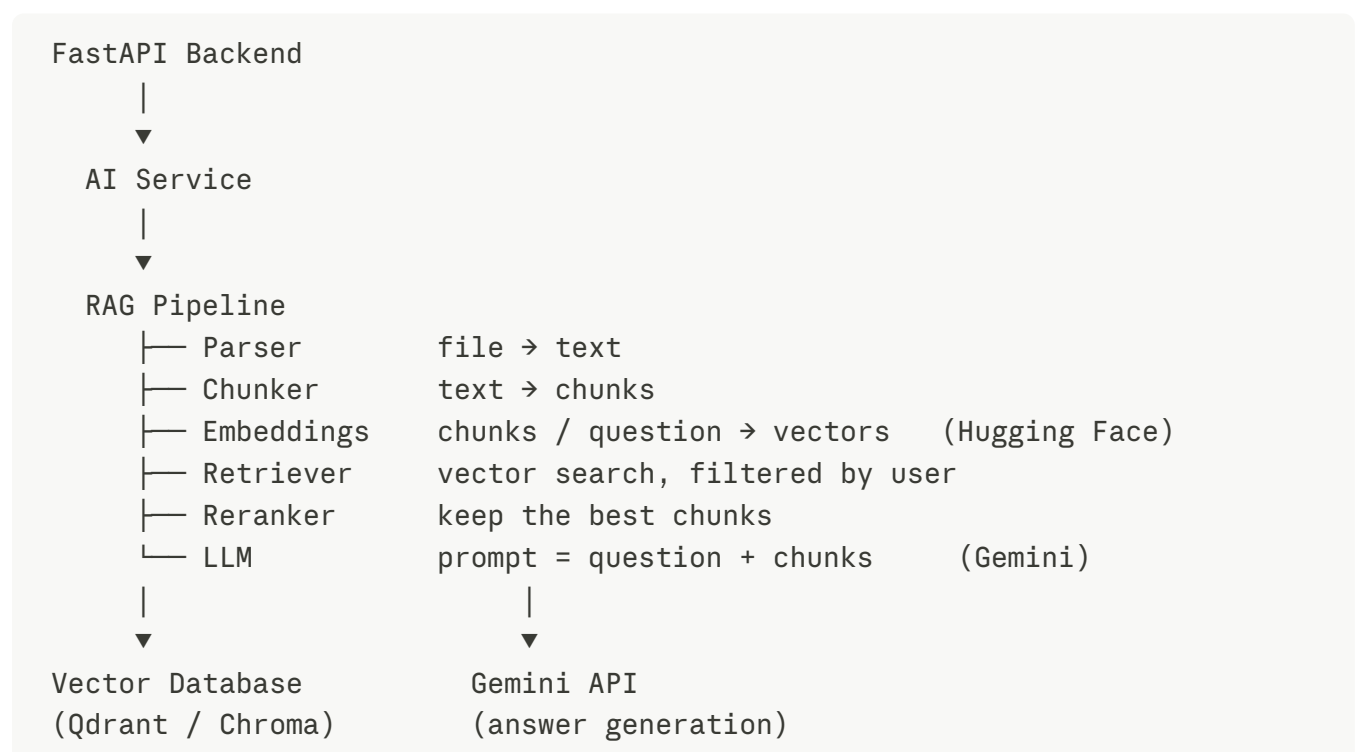
- *Route* = the door. *Schema* = the form at the door. *Service* = the decision-maker. *Repository* = the filing clerk. *Model* = the filing cabinet's layout. *Core* = the building's plumbing and electricity.
 - Dependencies flow **downward only**: routes call services, services call repositories, repositories use models. Never the reverse.
 - If you can't decide where code goes, ask: *Is it about HTTP, rules, or data access?*
-

12. Final Architecture Diagrams

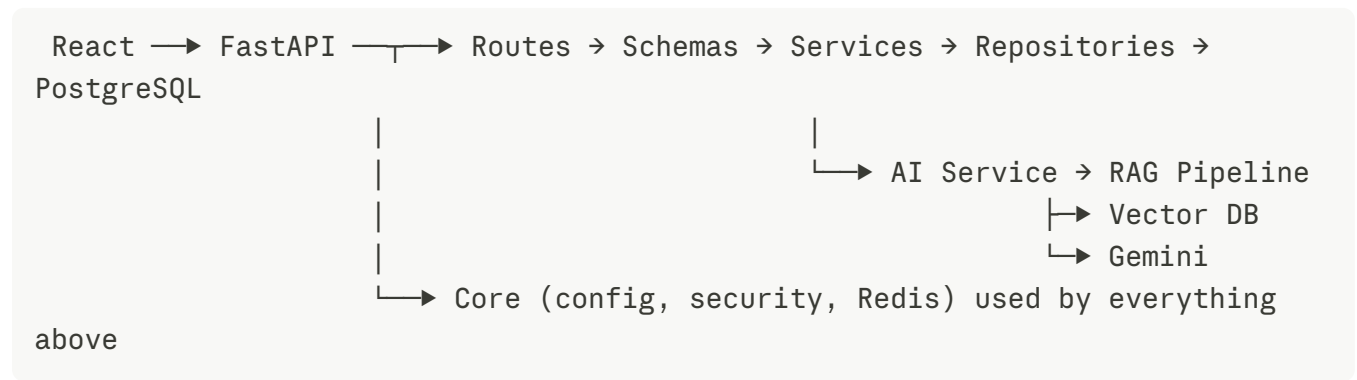
12.1 Application architecture



12.2 AI/ RAG architecture



12.3 Both together (one-glance view)



Revision Checklist

Before an interview, make sure you can explain, out loud and without notes:

- ☐ Why there is **no single correct** FastAPI structure, and what is convention vs requirement
- ☐ Route vs Service, Schema vs Model, Service vs Repository, Config vs Business Logic
- ☐ The request path: Route → Schema → Service → Repository → SQLAlchemy → PostgreSQL, and back
- ☐ Login flow, JWT, and `Depends(get_current_user)`
- ☐ Authentication (who) vs authorization (what is allowed)
- ☐ Where file metadata and file bytes are stored, and why
- ☐ Indexing pipeline vs query pipeline in RAG
- ☐ Why the LLM does not search the vector DB; the application does
- ☐ PostgreSQL vs vector DB responsibilities
- ☐ Three beginner mistakes you would avoid, and why

Final reminder: start simple. Add structure when the code asks for it, not before.